

评委一评分，签名及备注	队号： 10302	评委三评分，签名及备注
评委二评分，签名及备注	选题： A:2048	评委四评分，签名及备注

题目：基于 Monte Carlo 局面评估和 UCT 博弈树搜索的 2048

摘要

本文首先提出 Random-Max-Trees 算法来实现人工智能的 2048。此算法是通过静态评估函数来求得最优解。但是在实现的过程中出现冗余的现象，当移动方格步数过多的时候，好的评估函数却很难找到，使 Random-Max-Trees 算法效率降低。随后本论文采用 Alpha-Beta 算法，是前者的一种改进，在搜索结点数一样的情况下，可以使搜索深度达到原来的两倍。在实现的过程中发现 Alpha-Beta 严重依赖于着法的寻找顺序。只有当程序挑最好的子节来当先搜索，才会接近于实际分枝因子的平方根，也是该算法最好的状态。但是在首先搜索最坏的子节时，Beta 截断不会发生，此时该算法就如同 Random-Max-Trees 一样，效率非常低，也失去 Alpha-Beta 的优势，也无法试图通过面的搜索来弥补策略上的不足。

本文采用蒙特卡洛评估对以上模型进行了改进。它通过对当前局面下的每个的可选点进行大量的模拟来得出相应的胜负的统计特性，在简单情况下，胜率较高的点就可以认为是较好的点予以选择。由于 UCT 算法能不断根据之前的结果调整策略，选择优先评估哪一个可下点。所以在蒙特卡洛德基础上运用 UCT 算法提高收敛速度。可求得概率为 100%。

对于第二问，采用归纳法以及概率论量化数值，当方格为 4×4 时，最大能达到 $2^{4^2+1}=131072$ ，如果将方格扩展到 $N \times N$ 个，能达到的最大数为 2^{n^2+1} 。最后对模型进行评价。

本论文算法是采用 JAVA、C++ 以及 MATLAB 实现。

关键字：Random-Max-Trees；Alpha-beta；Monte Carlo；UTC；概率论

基于 Monte Carlo 局面评估和 UCT 博弈树搜索的 2048

1. 问题提出

2048 是最近一款非常火爆的益智游戏，很多网友自称“一旦玩上它就根本停不下来”。2048 游戏的规则很简单：每次控制所有方块向同一个方向运动，两个相同数字的方块撞在一起之后合并成为他们的和，每次操作之后会在空白的方格处随机生成一个 2 或者 4，最终得到一个“2048”的方块就算胜利了。如果 16 个格子全部填满并且相邻的格子都不相同也就是无法移动的话，那么游戏就会结束。

本文将建立数学模型，解答下列问题：

1. 如何才能达到 2048，给出一个通用的模型，并采用完成游戏所需移动次数和成功概率两个指标来验证模型的有效性；
2. 得到 2048 之后，游戏还可以继续玩，那么最大能达到多大的数值呢？如果将方格扩展到 $N \times N$ 个，能达到的最大数是多少？

2. 问题分析

本文首先基于 Random-Max-Trees 和 Alpha-beta 剪枝算法来实现人工智能(AI)的 2048。本文认为可以把 2048 游戏看成是一场人类和计算机的博弈，人类控制所有方块向同一个方向移动并合并，计算机则在空白处随机放置一个“2”或“4”的方块。但是在 AI 的环境下，博弈双方都是计算机，双方都不理性，所以在 AI 的环境下，选用更加保守的 Random-Max-Trees 博弈策略比选用 Mini-Max-Tree 的更加适当。如果把当前格局作为博弈树的父节点，把下一步所有可能的走法所造成的格局作为树的一个子节点，如果继续使用 Random-Max-Trees 算法，则此算法的效率很不理想，会造成许多不必要的步骤。因为每一个子节后面还有子节，可能的情况很多循环往复，直到 2048 为止，但是并不是所有的节点都必须搜索完毕，有些节点是不必要的。为了解决这一问题，本文可以采用 Alpha-beta 剪枝算法。

对于第一问为实现 2048 的这种情况，蒙特卡洛评估是一个很好的解决方法，它通过对当前局面下的每个的可选点进行大量的模拟来得出相应的胜负的统计特性，在简单情况下，胜率较高的点就可以认为是较好的点予以选择。UCT 在蒙特卡洛评估算法的基础上很大的提高了收敛速度，UCT 能不断根据之前的结果调整策略，选择优先评估哪一个可下点，基于此策略从而也可以得到第二问的答案。

对于第二问，如何计算出最大值是有条件的，对有随机值的问题，我们需要量化数值，如果考虑随机那么就有不确定性，这会使我们无法接近最大值，所以在最开始就直接规定好，取 4 来作为加数，因为这是累计的题目，4 比 2 大，所以在有限的范围内，4 的总和肯定比 2 的总和大。规定好数值，开始计算最大值。最大值是 2 的倍数，通过玩 2048 已经知道相同的数可以合并形成更大的数，合并一个数需要有两个数的空间，通过这一规律我们可以知道当格子都排满的情况下，还

是有可能进行数字的合并的，如果可以进行合并的话，那就又增加一个 4，这时又是满格的情况，如果合并过的数再次进行和并又再次产生一个空位，那么以此类推用过可以产生 15 次空格，那么，这 15 有同时增加了 15 次 4，通过这些新加入的 4，数值会进一步的增加。

3. 模型的假设

- 1) Alpha-Beta 有比 Minimax 搜索算法更加准确的评估函数才能保证那些优秀的节点不被过早的剪枝；
- 2) 博弈树较小而可以被完全展开，博弈树叶子节点的价值可以通过胜负关系来确定，搜索的结果就是最优解；
- 3) 博弈树很大而不能被完全展开时，博弈树叶子节点的价值可以通过静态评估函数计算出来，当静态评估函数较为准确时，就可以得到相应的近似最优解；

4. 符号说明

D: 2048 中块数的最大数值

N: 无数字块数

$I(x)$: minimax 的最小值

$F_i(x)$: 定义在 R^n 上的实值函数

$\overline{X_j}$: 第 j 台机器到目前为止的平均收益

$T(n)$: 第 j 台机器被测试的次数

n : 所有机器目前被测试的总次数

5. 型建立与求解

5.1. 问题一

5.1.1 Random-Max-Trees 算法

Random-Max-Trees 是从 Minimax 算法改变而来。只是加了一个条件，双方都是非理性的条件下，本质还是 Minimax。

Minimax 的定义

$$I(x) = \{i \mid f_i(x) = F(x)\}$$

$$\min_x F(x) = \max_{i \leq i \leq m} f_i(x), x \in R^n, \quad (1.1)$$

$f_i(x), i = 1, 2, \dots, m$ 是定义在 R^n 上的实值函数。它等价于非线性优化问题

$$\min z, \quad (1.2)$$

$$\text{s.t. } f_i(x) - z \leq 0, \quad i = 1, 2, \dots, m \quad (1.3)$$

可用求解约束问题的算法求解问题 (1.2) - (1.3)，从而得到 Minimax 问题的解。

假设在博弈过程中，对方总是选择使得博弈值最小的移动，我方则会选择博弈值最大的移动，对方称为 Min，则我方称为 Max，且都是非理性的。由于博弈双方是交替移动的，所以博弈树的结点及其父结点分属于我方与对方其中的一个，其种类分属 Max 和 Min。博弈树上的每个结点对应于一个深度，叶结点的深度为 0。因此，在任意的结点 node，对博弈双方均最优的博弈值为

$$\text{MinMax}(\text{node}) = \begin{cases} \text{Evaluate}(\text{note}) \\ \text{MAX}_{\text{seSuccessors}(\text{node})}(\text{MinMax}(\text{s})) \\ \text{MIN}_{\text{seSuccessors}(\text{node})}(\text{MinMax}(\text{s})) \end{cases}$$

由此，很自然地得出 Random-Max-Trees 算法，用来求出满足一些条件的二人零和博弈问题的博弈值。

5.1.2. Alpha-beta 剪枝算法

Alpha-beta 算法是对 Minimax 算法的优化，运行效率比 Minimax 更高，自然也比 Random-Max-Trees 快。Random-Max-Trees 算法在检查整个博弈树，效率非常低的，当步数越来越多的时候，每次搜索更深一层时，树的大小就呈指数式增长，工作量越来越大，存在着 2 种明显的冗余现象，自然降低了搜索的效率。第 1 种现象是极大值冗余。在图 1-a 中，节点 A 的值应是节点 B 和节点 C 的值中之较大者。现在已知节点 B 的值大于节点 D 的值。由于节点 C 的值应是它的诸子节点的值中之极小者，此极小值一定小于等于节点 D 的值，因此亦一定小于节点 B 的值，这表明，继续搜索节点 C 的其他诸子节点 E, F, ... 已没有意义，它们不能做任何贡献，于是把以节点 C 为根的子树全部剪去。这种优化称为 Alpha 剪枝。在图 1-b 是与极大值冗余对偶的现象，称为极小值冗余。节点 A 的值应是节点 B 和节点 C 的值中之较小者。现在已知节点 B 的值小于节点 D 的值。由于节点 C 的值应是它的诸子节点的值中之极大者，此极大值一定大于等于节点 D 的值，因此也大于节点 B 的值，这表明，继续搜索节点 C 的其他诸子节点已没有意义，并可以把以节点 C 为根的子树全部剪去，这种优化称为 Beta 剪枝。

而 Alpha-beta 算法是在众多路线里尽可能选择最好的线路。要想通过检查搜索树的前面几层，并且在叶子结点上用启发式的评价，那么做尽可能深的搜索是很重要的。下面通过比较来进一步了解 Random-Max-Trees 算法与 Alpha-beta 剪枝算法的关系。

A. 对于一个 Min 节点，若能估计出其倒推值的上确界 Beta，并且这个 Beta

值不大于 Min 的父节点 (Max 节点) 的估计倒推值的下确界 Alpha, 即 $\text{Alpha} \geq \text{Beta}$, 则就不必再扩展该 Min 节点的其余子节点了, 因为这些节点的估值对 Min 父节点的倒推值已无任何影响了, 这一过程称为 Alpha 剪枝。

- B. 对于一个 Max 节点, 若能估计出其倒推值的下确界 Alpha, 并且这个 Alpha 值不小于 Max 的父节点 (Min 节点) 的估计倒推值的上确界 Beta, 即 $\text{Alpha} \geq \text{Beta}$, 则就不必再扩展该 Max 节点的其余子节点了, 因为这些节点的估值对 Max 父节点的倒推值已无任何影响了。这一过程称为 Beta 剪枝。
- C. 一个 Max 节点的 Alpha 值等于其后继节点当前最大的最终倒推值, 一个 Min 节点的 Beta 值等于其后继节点当前最小的最终倒推值

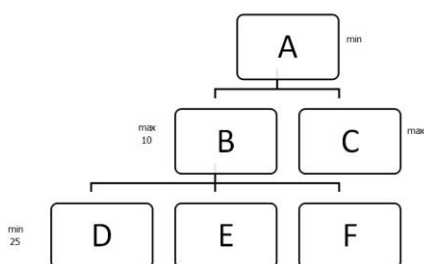


图 1-a

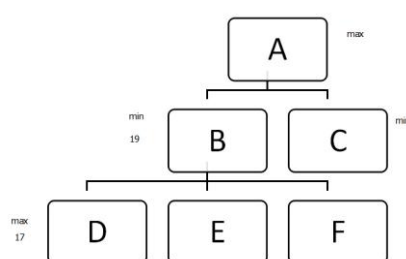


图 1-b

采用 Alpha-beta 剪枝, 可以在相同时间内加大 Random-Max-Trees 的搜索深度, 因此可以获得更好的效果。

5.1.3. 问题一模型的建立与求解

1) 本论文对 2048 游戏进行抽象化表述:

我方: (即游戏玩家) 每次可以选择上、下、左、右四个行棋策略中的一种 (某些格局会少于四种, 因为有些方向不可走)。行棋后方块按照既定逻辑移动及合并, 格局转换完成。

对方: (计算机) 在当前任意空格子里放置一个方块, 方块的数值可以是 “2” 或 “4”。放置新方块后, 格局转换完成。

胜利条件: 出现某个方块的数值为 “2048”。

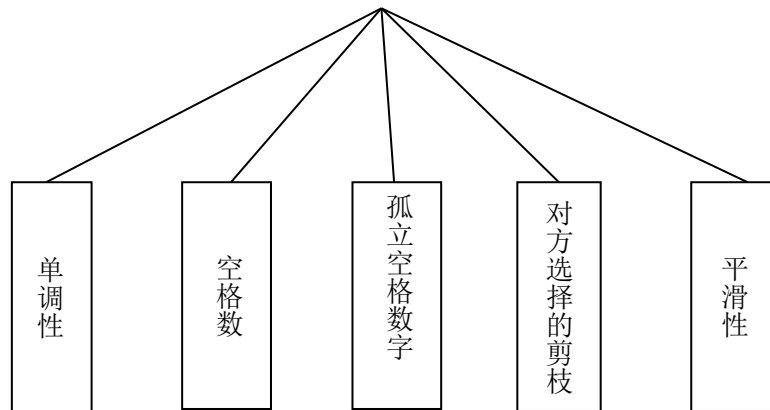
失败条件: 格子全满, 且无法向四个方向中任何一个方向移动 (均不能触发合并)。

这样分析, 于是 2048 游戏就可化成建立一个模型解决信息对称的双人对弈问题。

2) 评价当前格局的价值

在 2048 中, 除了终局外, 中间格局并无非常明显的价值评价指标, 因此需要用一些启发式的指标来评价格局。那些分数高的 “好” 格局是容易引向胜利的格局, 而分低的 “坏” 格局是容易引向失败的格局。

本文采用了如下几个启发式指标，如下：



解释：

(1) 单调性

单调性指方块从左到右、从上到下均遵从递增或递减。一般来说，越单调的格局越好。

(2) 平滑性

是指每个方块与其直接相邻方块数值的差，其中差越小越平滑。例如 2 旁边是 4 就比 2 旁边是 128 平滑。一般认为越平滑的格局越好。

(3) 空格数

这个很好理解，因为一般来说，空格子越少对玩家越不利。所以我们认为空格越多的格局越好。

(4) 孤立空格数

这个指标评价空格被分开的程度，空格越分散则格局越差。

(5) 对方选择的剪枝

在这个程序中，除了采用 Alpha-beta 剪枝外，在 Min 节点还采用了另一种剪枝，即只考虑对方走出让格局最差的那一步（而实际 2048 中计算机的选择是随机的），而不是搜索全部对方可能的走法。这是因为对方所有可能的选择为“空格数 $\times 2$ ”，如果全部搜索的话会严重限制搜索深度。

3. 可以找实验玩家获得数据，再进行因子分析，根据这五种因素对到达 2048 步数的影响确定权重。

5.1.4. 问题一模型的改进

5.1.4.1 蒙特卡罗算法

蒙特卡罗(Monte Carlo)方法也称为随机模拟方法，是一种最优有限的搜索方法。它的基本思想是，为了求解数学、物理、工程技术以及生产管理等方面的问题，首先建立一个概率模型或随机过程，使它的参数等于问题的解；然后通过对模型或过程的观察或抽样试验来计算所求参数的统计特征，最后给出所求解的近似值。

在应用蒙特卡罗方法解决 2048 问题的过程中，大体上有如下几个内容：

1) 对求解的问题建立简单而又便于实现的概率统计模型，使所求的解恰好是所建

- 立模型的概率分布或数学期望。
- 2) 根据概率统计模型的特点和计算实践的需要, 尽量改进模型, 以便减小方差和降低费用, 提高计算效率。
 - 3) 建立对随机变量的抽样方法, 其中包括建立产生伪随机数的方法和建立对所遇到的分布产生随机变量的随机抽样方法。
 - 4) 给出获得所求解的统计估计值及其方差或标准误差的方法。

5.1.4.2 UCT 算法(UCB for tree search)

UCT 又名 UCB for Tree Search, 是上限信心界(Upper confidence bound,UCB)在 Tree Search 上的应用。UCB 策略是为了解决相互独立且收益率不同的因子, 获得尽可能大的回报的最佳策略。大致上来说, 每一次操作 UCB 会根据每一个因子目前的平均收益值, 加上一个额外的参数, 得出本次操作此因子的 UCB 值, 然后根据此值, 挑选出拥有最大 UCB 值的因子, 作为本次操作所要选择的因子。其中, 所谓额外参数, 会随每一台因子被选择的次数增加而相对减少, 其目的在于让选择因子时, 不过分拘泥于旧有的表现, 而可以适度地探索其他因子。

UCB 公式的一般型式:

$$Score = Exploitation + Exploration$$

在实践中, 检定效果较好的一个 UCB 公式表示如下:

$$\overline{X}_j + \sqrt{\frac{2 \log n}{T_j(n)}}$$

$\overline{X}_j$ 是第 j 台机器到目前为止的平均收益, $T_j(n)$ 是第 j 台机器被测试的次数, n 是所有机器目前被测试的总次数。

Tree Search 开始时, UCT 会建立一棵 Tree, 然后:

- 1) 从根节点开始
- 2) 利用 UCB 公式计算每个子节点的 UCB 值, 选择 UCB 值最高的子节点
- 3) 若此子节点并非叶节点(从未拜访过的节点), 则由此节点开始, 重复(2)
- 4) 直到遇到叶节点, 则计算叶节点的收益值, 并依此更新根节点到此一节点路径上的所有收益值
- 5) 由(1)开始重复, 直到时间结束, 或达到某一预设次数
- 6) 由根节点的所有子节点中, 选择平均收益值最高者, 作为最佳节点, 此一节点, 就是 UCT 的结果。

5.1.4.3 改进的模型

其中 a_i 代表第 i 个因子, v_i 表示第 i 个因子目前被访问的次数, w_i 表示第 i 个因

子目前获胜的次数，其中 $i \in [1, k]$ ，我们可以知道， $v_i \geq w_i$ 并且 $v \geq w$ ，这是因为获胜次数永远受到访问次数的限制。先用 $v = \sum_i v_i$ 表示到目前为止所有手臂被访问

的次数和， $w = \sum_i w_i$ 表示到目前为所有因子的获胜次数和。显然地，如果 $v_j > \frac{v}{2}$ ，，

则 $\sum_{i \neq j} v_i = V - v_j < \frac{V}{2}$ ，因此有 $v_i < \frac{V}{2} < v_j$

当利用 UCT 算法确定究竟哪一个可下点会变成最终的落子点时，总是会选择那个被访问最大次数的可下点。所以， $j \in \{1, \dots, k\} \cap i \neq j$ 利用绝对剪枝条件时，被访问最多次数的节点绝对不可能满足剪枝条件，这样就保证在使用绝对剪枝条件后，根据访问次数做出的最终决策结果将和使用原始的 UCT 方法保持一致。

如果存在明显好于其他节点的可下点，总被访问次数可能在没有达到我们预期的总访问次数时就已经有节点的访问次数突破预计访问次数的一半。因此，在加入绝对剪枝条件后，可以提前终止模拟，达到节省时间的作用。

5.1.4.4 模型的验证

分析：移动次数和成功概率两个指标来验证模型的有效性，需要单独提出移动次数和成功概率进行分析，移动次数和成功概率可以通过程序进行实践，程序内部含有移动次数的统计和制定的目标既是移动数值达到 2048，自动移至 2048 是用到了 AI，自动的进行实践，AI 能够自动且合理的规避无法移至 2048 的可能，使达到 2048 成为可能。

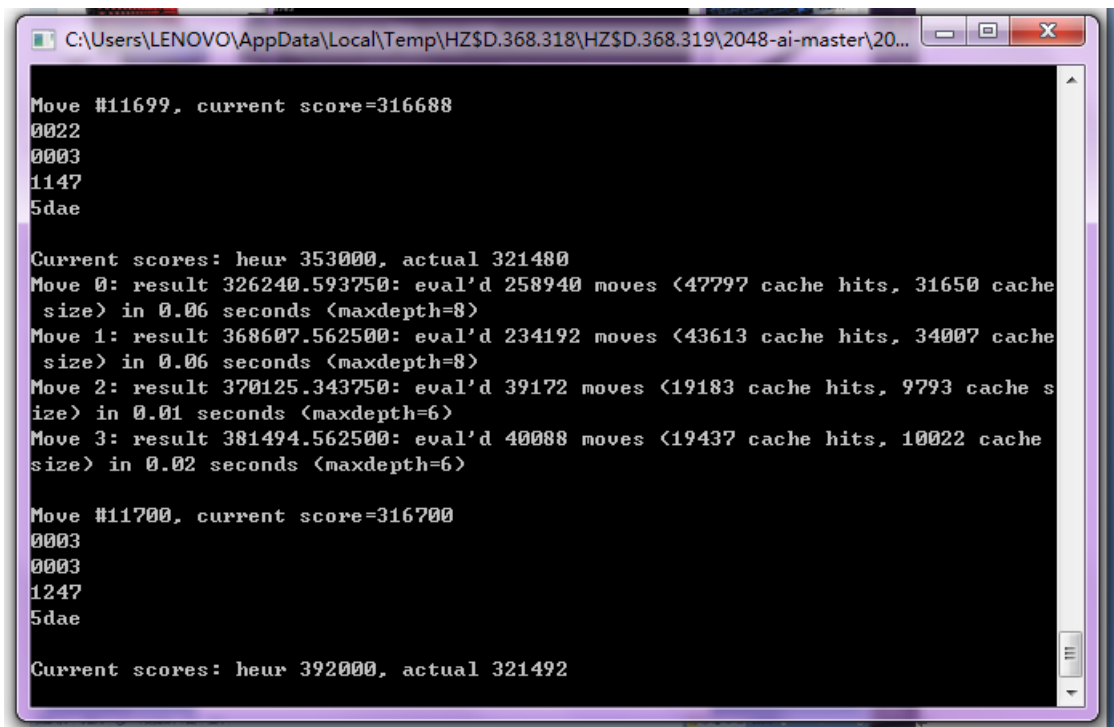
证明：

移动次数是通过 AI 来进行统计，AI 会找到最合理的路径，虽然不会是最短的路径，却是最合理的路径。

```
static inline board_t execute_move(int move, board_t board) {
    switch(move) {
        case 0: /* up*/ return execute_move_0(board);
        case 1: /* down*/return execute_move_1(board);
        case 2: /*left*/return execute_move_2(board);
        case 3: /*right*/return execute_move_3(board);
        default:return ~0ULL;
    }
}
```

这是 AI 的方向移动，switch 选择方向，move 由别的函数赋值来对 AI 进行路线的选择，从中找到最合理的路线。

最合理的路线是表示能够在成功合并成 2048 的前提下分数最高，且路径也少，通过程序的运行能够输出结果：



```
C:\Users\LENOVO\AppData\Local\Temp\HZSD.368.318\HZSD.368.319\2048-ai-master\20...

Move #11699, current score=316688
0022
0003
1147
5dae

Current scores: heur 353000, actual 321480
Move 0: result 326240.593750: eval'd 258940 moves <47797 cache hits, 31650 cache
size> in 0.06 seconds <maxdepth=8>
Move 1: result 368607.562500: eval'd 234192 moves <43613 cache hits, 34007 cache
size> in 0.06 seconds <maxdepth=8>
Move 2: result 370125.343750: eval'd 39172 moves <19183 cache hits, 9793 cache s
ize> in 0.01 seconds <maxdepth=6>
Move 3: result 381494.562500: eval'd 40088 moves <19437 cache hits, 10022 cache
size> in 0.02 seconds <maxdepth=6>

Move #11700, current score=316700
0003
0003
1247
5dae

Current scores: heur 392000, actual 321492
```

可以从上图的结果中看到，当玩到 2048 时一共移动了 11700 步，分数是 316700，但是这个结果并不是唯一的，因为每一次开始的数值都是不一样的，那么 AI 对于不同的情况给出的操作也是不同的，所以这也是答案不唯一的原因。但是通过多次的模拟总结，发现移动的次数是离散的，散点状的分布在 10000 到 15000 之间，这个我们可以用函数分部的方式来进行分析。

AI 能够 100%的运行并成功达到 2048，所以这个成功率是 100%的。

5.2 问题二模型的建立与求解

本文发现2048中每一格都是 $2n$ 的整数，通过相同 $2n$ 的数块结合形成 $2n+1$ 的数块，同时在移动的过程中产生新的数块，包括21和22两种数块，且是随机均等几率的。

形成一个22需要2个21，且需要两个数块来形成一个数块。

形成一个23需要2个22，如果都是22的情况则需要2个数块形成一个数块，同理如给出1个22的情况和2个21的情况同时存在时，需要三个数块形成一个数块。

同理可知24, 25直至 $2n$ 次，可以用2个 $2n-1$ 来形成。

假设在16格中存在最大数 X ，且环境最为理想

则假设 $X=2n$

形成一个 $2n$ 两个 $2n-1$ ，作为最理想的情况来求解

假设已经存在一个 2^{n-1} ，另一个 2^{n-1} 便由2个 2^{n-2} 形成，同时已经存在一个 2^{n-2} ，那么另一个 2^{n-2} 由2个 2^{n-3} 形成，以此类推，我们可以发现每一个2次数级都拥有一个数块，而2048这款游戏有4x4的16个数块，那么我们可以推断出16个数块存在16个不同的数级。因为移动中随机产生的数有2和4，那么我们便独立讨论

当数从21开始，那么最高数值便是 $21+15=216$

2	4	8	16
256	128	64	32
512	1024	2048	4096
8192	16384	32768	65536

当数从22开始，那么最高数值便是 $22+15=217$

4	8	16	32
512	256	128	64
1024	2048	4096	8192
131072	65536	32768	16384

作为理想化的推断，以每次都是4出现的话，可知217为最高数值。

$X=2$ 或者4，假定之后一直都是选定的数，作为理想情况。

N 为剩下的数块数

D 为最大数值

$X=4$ 的情况

初始情况为2个2时，可合并成4，此时产生一个4，剩下14个数块

移动生成8，再次产生一个4，剩下一14个数块，D 为8

随机移动，产生一个4，剩下13个数块，D 为8

再次移动产生4，同时合并4，产生2个8，剩下13个数块，D 为8

再次移动产生4，合并产生16，剩下14个数块，D 为16

以此类推我们可以发现，当数块为奇数个时必剩下4

D=错误!未找到引用源。

当 $X=2$ 的情况，则可以推断出：

D=错误!未找到引用源。

如果将方格扩展到 $N \times N$ 个，能达到的最大数是多少？

使用数学归纳法来证明。

之前已经证明出来，在理想的环境下， 4×4 的 16 格内最大的数值是 217，

而当只有一个数块的时候，最大值的情况只有 4

可证 当 n 取 1 时， $f(1)_{\max}=4$

当 n 取 4 时， $f(4)_{\max}=131072$

假设当 $n=k$ 时 ($k \geq n_0$, k 为自然数)，命题成立。

证明 当 $n=k+1$ 时命题同样成立。

当 $n=4$ 时，D=错误!未找到引用源。

所以当 $n=k$ 时，D=错误!未找到引用源。

当 $n=k$ 成立，得出 D=错误!未找到引用源。

则， $n=k+1$ 时，D=错误!未找到引用源。

错误!未找到引用源。—错误!未找到引用源。(错误!未找到引用源。)

=错误!未找到引用源。—错误!未找到引用源。

=错误!未找到引用源。—错误!未找到引用源。

=错误!未找到引用源。

上述可证：

D=错误!未找到引用源。

当只有一个数块时取 $k=0$ 作为特殊情况，其余时候， k 为给定值

当 2×2 时：D=错误!未找到引用源。=32

4	8
---	---

32	16
----	----

当 3×3 时：D=错误!未找到引用源。 =1024

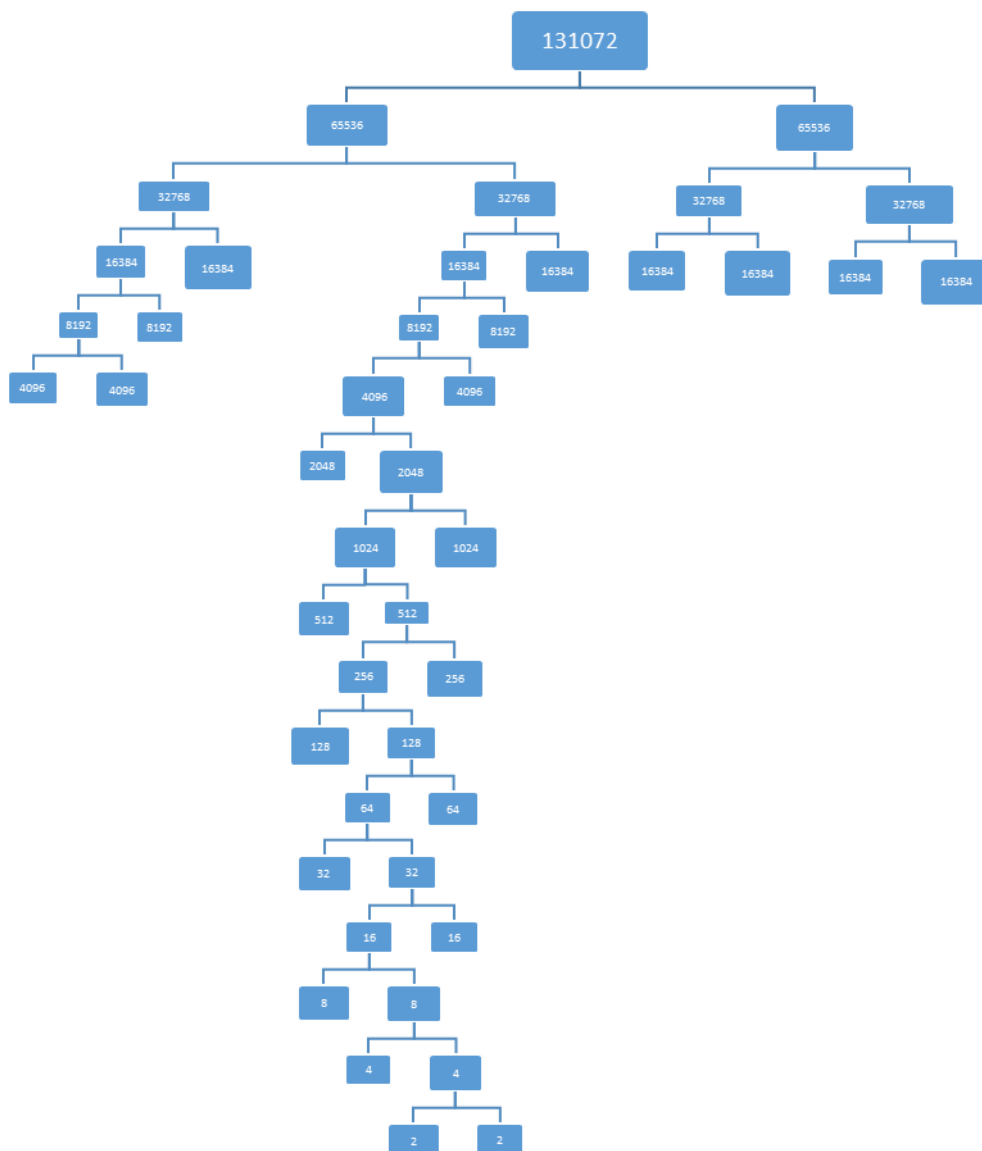
4	8	16
128	64	32
256	512	1024

当 4×4 时：D=错误!未找到引用源。 =131072已经证明了

所以以此类推可知：

$N \times N$ =错误!未找到引用源。

验证程序见附录。



6. 模型的评价

① 模型的优点

- 1) 博弈树叶子节点的价值可以通过胜负关系来确定，最优解明显；
- 2) 搜索深度大，搜索速度快；
- 3) 从问题出发，建立了数学模型，用大量的数据进行验证，实现 2048 提供了理论依据。

② 模型的缺点

- 1) 在第一问中，博弈树往往很大，而好的评估函数却很难找到，制约了极小极大算法的应用范围；
- 2) 在第一问中，通过全面的搜索来弥补策略上的不足，不太可能；
- 3) 本文模型的建立和求解几乎只是从统计的角度出发的。

7. 参考文献

- [1] 周明明, UCT 算法在计算机围棋中的应用与改进 2012 (增刊): 330-06
- [2] 刘宇, Monte_carlo 方法在计算机围棋中的应用 47 (12)
- [3] 岳金朋, 中国象棋 Alpha_Beta 搜索算法的研究与改进 2009-04 45(2)
- [4] 王晖, 基于高阶差分方法半参数回归模型参数估计及其 minimax 性质 [D]; 华中师范大学;2012 年
- [5] 张加佳-基于 UCT 算法的非完备信息多人军旗博弈系统 哈尔滨工业大学 2008 年 12 月
- [6] 闫冰-IMRT 中基于 MonteCarlo 模拟数据的 C_S 算法研究合肥工业大学
- [7] 吴哲-关于 Roesser 离散状态空间模式 《东北大学学报(自然科学版)》1982.
- [8] 吕志明-基于符号对象函数的离散状态空间方程求解 电气电子教学学报
- [9] 杨成梧-一般 2_D 线性常系数离散状态空间_省略_稳定性的一类 Lyapunov 方法[J];南京理工大学学报;2000 年 02 期
- [10] <http://www.zhihu.com/question/23141069>

8. 附 件

%Random-Max-Trees Java 代码

```
public class MinMaxTree
{

    /// <summary>
    ///
    /// </summary>
    /// <param name="deep">深度</param>
    /// <param name="tree">ITree</param>
    /// <returns>ITree</returns>
    public static ITree Max(int deep, ITree tree)
    {
        //定义一个 bestTree 用于存放参数 tree 的子节点中，权重最大的一个节点
        ITree bestTree = null;

        if (deep > 0)
        {
            deep--;
            //循环判断子节点的权重
```

```

        foreach (ITree t in tree.list)
        {
            //it 用来存放当前节点 t 的子节点
            //获取子节点返回的最小权重权重值的节点
            ITree it = Min(deep, t);

            if (bestTree == null)
            {
                //将权重值赋值给 t, 然后将 t 的同级节点, 赋值给 bestTree
                t.weight = it.weight;
                bestTree = t;
            }
            else
            {
                //和下一个节点比较, 找出权重最大的一个节点
                if (bestTree.weight < it.weight)
                {
                    t.weight = it.weight;
                    bestTree = t;
                }
            }
        }

        //返回 bestTree
        return bestTree;
    }

    return tree;
}

/// <summary>
/// Min
/// </summary>
/// <param name="deep">深度</param>
/// <param name="tree">ITree</param>
/// <returns>ITree</returns>
    public static ITree Min(int deep, ITree tree)
    {
        ITree bestTree = null;
        if (deep > 0)
        {
            deep--;
            foreach (ITree t in tree.list)
            {
                ITree it = Max(deep, t);
            }
        }
    }
}

```

```

        if (bestTree == null)
        {
            t.weight = it.weight;
            bestTree = t;
        }
        else
        {
            if (bestTree.weight > it.weight)
            {
                t.weight = it.weight;
                bestTree = t;
            }
        }
    }
    return bestTree;
}

return tree;
}
}

% Alpha-beta 剪枝算法代码
int AlphaBeta(int depth, int alpha, int beta)
{
    //如果层数为 0 或者已达最终状态则返回本步棋的估值
    if(depth == 0 || IsGameOver()) return Evaluate();
    for(each possible move){
        MakeMove();
        int val = -AlphaBeta(depth - 1, -beta, -alpha);
        UnMakeMove();
        if(val >= beta){
            return val;
        }
        if(val > alpha){
            alpha = val;
        }
        ...
    }
    return alpha;//返回最好的值
}

%UCT 搜索的代码:
Orientation UctPlayer::NextMove(const FullBoard& full_board) const {
    int mc_count = 0;
    while (mc_count < kMonteCarloGameCount) {
        FullBoard current_node;

```

```

        Orientation orientation = MaxUcbMove(full_board);
        current_node.Copy(full_board);
        current_node.PlayMovingMove(orientation);
        NewProfit(current_node, &mc_count);
    }

    return BestChild(full_board);
}

%2048
/*2048*/
#include<stdio.h>
#include<windows.h>
#include<stdlib.h>
#include<conio.h>
#include<time.h>

int code[4][4]={0,0,0,0,
                0,0,0,0,
                0,0,0,0,
                0,0,0,0};/*游戏中的 16 个格子*/
int temp[5];/*中间变量*/
int move=0;/*移动次数*/
int score=0;/*分数*/

void print(void)/*显示游戏界面*/
{
    int i,j;

    system("CLS");/*清屏*/
    printf("2048\n");
    printf("W--UP  A--LEFT  S--DOWN  D--RIGHT  0--EXIT\n");
    printf("Score:%d Move:%d\n",score,move);
    printf("Made by Yanjisheng\n");
    printf("|-----|\n");/*显示横向分隔线*/
    for(i=0;i<=3;i++)
    {
        for(j=0;j<=3;j++)
        {
            if(code[i][j]==0)
            {
                printf("|    ");/*0 显示空格*/
            }
            else
            {

```

```

        printf("|%4d",code[i][j]);/*显示数字和分隔线*/
    }
}
printf("\n|-----|\n");/*显示横向分隔线*/
}
}

int add(void)/*对中间变量数组进行处理*/
{
    int i;
    int t=0;
    int change=0;/*判断数组是否有改变，0 不变，1 变化*/

    do
    {
        for(i=0;i<=3;i++)
        {
            if(temp[i]==0)
            {
                if(temp[i]!=temp[i+1])
                    change=1;/*当一个 0 后面不是 0 时数组改变*/
                temp[i]=temp[i+1];
                temp[i+1]=0;
            }
        }/*去掉中间的 0*/
        t++;
    }while(t<=3);/*重复多次*/
    for(i=1;i<=3;i++)
    {
        if(temp[i]==temp[i-1])
        {

            if(temp[i]!=0)
            {
                change=1;/*当两个非零相同的数相加时数组改变*/
                score=score+temp[i];/*加分*/
            }
            temp[i-1]=temp[i-1]*2;
            temp[i]=0;
        }
    }/*把两个相邻的相同的数加起来*/
    do
    {
        for(i=0;i<=3;i++)

```

```

    {
        if(temp[i]==0)
        {
            temp[i]=temp[i+1];
            temp[i+1]=0;
        }
    }/*去掉中间的 0*/
    t++;
}while(t<=3);/*重复多次*/
return change;
}

int main(void)
{
    int gameover=0;/*判断游戏是否结束，1 结束，0 继续*/
    int i,j;
    int change=1;/*判断格子中的数是否改变，0 不变*/
    char input;

    srand((unsigned)time(NULL));/*设置随机数的起点*/
    while(gameover==0)
    {
        if(change>=1)/*仅当数发生改变时添加新数*/
        {
            do
            {
                i=((unsigned)rand())%4;
                j=((unsigned)rand())%4;
            }while(code[i][j]!=0);
            if(((unsigned)rand())%4==0)
            {
                code[i][j]=4;
            }
            else
            {
                code[i][j]=2;/*随机选一个空格填上 2 或 4*/
            }
            move++;/*增加次数*/
        }
        print();/*显示*/
        input=getch();/*输入方向*/

        change=0;
        switch(input)

```

```

{
    case '0':/*退出*/
        printf("Are you sure to exit?(y/n)");
        input=getchar();
        if(input=='y'||input=='Y')
            exit(0);
        break;

    case 'W':
    case 'w':/*上*/
        for(j=0;j<=3;j++)
        {
            for(i=0;i<=3;i++)
            {
                temp[i]=code[i][j];/*把一列数移到中间变量*/
            }
            temp[4]=0;
            change=change+add();
            for(i=0;i<=3;i++)
            {
                code[i][j]=temp[i];/*把处理好的中间变量移回来*/
            }
        }
        break;

    case 'A':
    case 'a':/*左*/
        for(i=0;i<=3;i++)
        {
            for(j=0;j<=3;j++)
            {
                temp[j]=code[i][j];/*把一行数移到中间变量*/
            }
            temp[4]=0;
            change=change+add();
            for(j=0;j<=3;j++)
            {
                code[i][j]=temp[j];/*把处理好的中间变量移回来*/
            }
        }

        break;

    case 'S':

```

```

case 's':/*下*/
    for(j=0;j<=3;j++)
    {
        for(i=0;i<=3;i++)
        {
            temp[i]=code[3-i][j];/*把一列数移到中间变量*/
        }
        temp[4]=0;
        change=change+add();
        for(i=0;i<=3;i++)
        {
            code[3-i][j]=temp[i];/*把处理好的中间变量移回来*/
        }
    }
    break;

case 'D':
case 'd':/*右*/
    for(i=0;i<=3;i++)
    {
        for(j=0;j<=3;j++)
        {
            temp[j]=code[i][3-j];/*把一行数移到中间变量*/
        }
        temp[4]=0;
        change=change+add();
        for(j=0;j<=3;j++)
        {
            code[i][3-j]=temp[j];/*把处理好的中间变量移回来*/
        }
    }
    break;
}
gameover=1;
for(i=0;i<=3;i++)
    for(j=0;j<=3;j++)
        if(code[i][j]==0)
            gameover=0;/*所有格子都填满则游戏结束*/

}
printf("Game over!\n");
getch();
return 0;
}

```

```

%2048 求 move 的步数以及达到的最大值
/*2048*/
#include<stdio.h>
#include<windows.h>
#include<stdlib.h>
#include<conio.h>
#include<time.h>

int code[4][4]={0,0,0,0,
                0,0,0,0,
                0,0,0,0,
                0,0,0,0};/*游戏中的 16 个格子*/
int temp[5];/*中间变量*/
int move=0;/*移动次数*/
int score=0;/*分数*/

void print(void)/*显示游戏界面*/
{
    int i,j;

    system("CLS");/*清屏*/
    printf("2048\n");
    printf("W--UP  A--LEFT  S--DOWN  D--RIGHT  0--EXIT\n");
    printf("Score:%d Move:%d\n",score,move);
    printf("Made by Yanjisheng\n");
    printf("|-----|\n");/*显示横向分隔线*/
    for(i=0;i<=3;i++)
    {
        for(j=0;j<=3;j++)
        {
            if(code[i][j]==0)
            {
                printf("   ");/*0 显示空格*/
            }
            else
            {
                printf("|%4d",code[i][j]);/*显示数字和分隔线*/
            }
        }
        printf("\n|-----|\n");/*显示横向分隔线*/
    }
}

int add(void)/*对中间变量数组进行处理*/

```

```

{
    int i;
    int t=0;
    int change=0; /*判断数组是否有改变，0 不变，1 变化*/

    do
    {
        for(i=0;i<=3;i++)
        {
            if(temp[i]==0)
            {
                if(temp[i]!=temp[i+1])
                    change=1; /*当一个 0 后面不是 0 时数组改变*/
                temp[i]=temp[i+1];
                temp[i+1]=0;
            }
        } /*去掉中间的 0*/
        t++;
    } while(t<=3); /*重复多次*/
    for(i=1;i<=3;i++)
    {
        if(temp[i]==temp[i-1])
        {

            if(temp[i]!=0)
            {
                change=1; /*当两个非零相同的数相加时数组改变*/
                score=score+temp[i]; /*加分*/
            }
            temp[i-1]=temp[i-1]*2;
            temp[i]=0;
        }
    } /*把两个相邻的相同的数加起来*/
    do
    {
        for(i=0;i<=3;i++)
        {
            if(temp[i]==0)
            {
                temp[i]=temp[i+1];
                temp[i+1]=0;
            }
        } /*去掉中间的 0*/
        t++;
    }
}

```

```

        }while(t<=3);/*重复多次*/
        return change;
    }

int main(void)
{
    int gameover=0;/*判断游戏是否结束，1 结束，0 继续*/
    int i,j;
    int change=1;/*判断格子中的数是否改变，0 不变*/
    char input;

    srand((unsigned)time(NULL));/*设置随机数的起点*/
    while(gameover==0)
    {
        if(change>=1)/*仅当数发生改变时添加新数*/
        {
            do
            {
                i=((unsigned)rand())%4;
                j=((unsigned)rand())%4;
            }while(code[i][j]!=0);
            if(((unsigned)rand())%4==0)
            {
                code[i][j]=4;
            }
            else
            {
                code[i][j]=2;/*随机选一个空格填上 2 或 4*/
            }
            move++;/*增加次数*/
        }
        print();/*显示*/
        input=getch();/*输入方向*/

        change=0;
        switch(input)
        {
            case '0':/*退出*/
                printf("Are you sure to exit?(y/n)");
                input=getchar();
                if(input=='y'||input=='Y')
                    exit(0);
                break;

```

```

case 'W':
case 'w':/*上*/
    for(j=0;j<=3;j++)
    {
        for(i=0;i<=3;i++)
        {
            temp[i]=code[i][j];/*把一列数移到中间变量*/
        }
        temp[4]=0;
        change=change+add();
        for(i=0;i<=3;i++)
        {
            code[i][j]=temp[i];/*把处理好的中间变量移回来*/
        }
    }
    break;

case 'A':
case 'a':/*左*/
    for(i=0;i<=3;i++)
    {
        for(j=0;j<=3;j++)
        {
            temp[j]=code[i][j];/*把一行数移到中间变量*/
        }
        temp[4]=0;
        change=change+add();
        for(j=0;j<=3;j++)
        {
            code[i][j]=temp[j];/*把处理好的中间变量移回来*/
        }
    }

    break;

case 'S':
case 's':/*下*/
    for(j=0;j<=3;j++)
    {
        for(i=0;i<=3;i++)
        {
            temp[i]=code[3-i][j];/*把一列数移到中间变量*/
        }
        temp[4]=0;
    }

```

```

        change=change+add();
        for(i=0;i<=3;i++)
        {
            code[3-i][j]=temp[i];/*把处理好的中间变量移回来*/
        }
    }
    break;

case 'D':
case 'd':/*右*/
    for(i=0;i<=3;i++)
    {
        for(j=0;j<=3;j++)
        {
            temp[j]=code[i][3-j];/*把一行数移到中间变量*/
        }
        temp[4]=0;
        change=change+add();
        for(j=0;j<=3;j++)
        {
            code[i][3-j]=temp[j];/*把处理好的中间变量移回来*/
        }
    }
    break;
}
gameover=1;
for(i=0;i<=3;i++)
    for(j=0;j<=3;j++)
        if(code[i][j]==0)
            gameover=0;/*所有格子都填满则游戏结束*/

}
printf("Game over!\n");
getch();
return 0;
}

```

```
C:\Users\LENOVO\AppData\Local\Temp\HZ$D.368.318\HZ$D.368.319\2048-ai-master\20...
Move #8220, current score=224000
115e
049a
0147
0125

Current scores: heur 325000, actual 227428
Move 0: result 353307.906250: eval'd 780568 moves <45258 cache hits, 39545 cache
size> in 0.12 seconds <maxdepth=8>
Move 1: result 361149.656250: eval'd 360596 moves <49083 cache hits, 31288 cache
size> in 0.07 seconds <maxdepth=8>
Move 2: result 314667.062500: eval'd 826696 moves <36943 cache hits, 31951 cache
size> in 0.12 seconds <maxdepth=8>
Move 3: result 356020.812500: eval'd 738248 moves <48483 cache hits, 39911 cache
size> in 0.12 seconds <maxdepth=8>

Move #8221, current score=224004
205e
019a
0447
1225

Current scores: heur 305000, actual 227436
```

```
C:\Users\LENOVO\AppData\Local\Temp\HZ$D.368.318\HZ$D.368.319\2048-ai-master\20...
467e
223b
0123
1001

Current scores: heur 381000, actual 234652
Move 0: result 393799.156250: eval'd 1704072 moves <97654 cache hits, 78525 cache
size> in 0.31 seconds <maxdepth=8>
Move 1: result 357323.968750: eval'd 1967960 moves <65348 cache hits, 57728 cache
size> in 0.34 seconds <maxdepth=8>
Move 2: result 371889.968750: eval'd 311364 moves <58936 cache hits, 45924 cache
size> in 0.08 seconds <maxdepth=8>
Move 3: result 389057.031250: eval'd 226596 moves <49727 cache hits, 37578 cache
size> in 0.06 seconds <maxdepth=8>

Move #8454, current score=231120
467e
223b
1123
0011

Current scores: heur 341000, actual 234652
Move 0: result 0.000000: eval'd 0 moves <0 cache hits, 0 cache size> in 0.00 sec
onds <maxdepth=0>
```

```
C:\Users\LENOVO\AppData\Local\Temp\HZ$D.368.318\HZ$D.368.319\2048-ai-master\20...

Move #8610, current score=234176
eb22
9511
4200
1001

Current scores: heur 367000, actual 237756
Move 0: result 402018.687500: eval'd 133244 moves (49911 cache hits, 30761 cache
size) in 0.05 seconds (maxdepth=8)
Move 1: result 349488.906250: eval'd 173992 moves (43032 cache hits, 29903 cache
size) in 0.05 seconds (maxdepth=8)
Move 2: result 377881.062500: eval'd 50352 moves (20943 cache hits, 12588 cache
size) in 0.02 seconds (maxdepth=6)
Move 3: result 368690.500000: eval'd 55588 moves (24030 cache hits, 13897 cache
size) in 0.02 seconds (maxdepth=6)

Move #8611, current score=234180
eb22
9512
4220
1000

Current scores: heur 364000, actual 237764
```

```
C:\Users\LENOVO\AppData\Local\Temp\HZ$D.368.318\HZ$D.368.319\2048-ai-master\20...

Move #8742, current score=235732
e634
9b21
7661
1110

Current scores: heur 236000, actual 239364
Move 0: result 348371.625000: eval'd 1513952 moves (79949 cache hits, 53385 cache
size) in 0.27 seconds (maxdepth=8)
Move 1: result 331928.437500: eval'd 3382888 moves (56427 cache hits, 49083 cache
size) in 0.52 seconds (maxdepth=8)
Move 2: result 340143.031250: eval'd 1654924 moves (86823 cache hits, 69469 cache
size) in 0.28 seconds (maxdepth=8)
Move 3: result 321545.093750: eval'd 1839456 moves (87671 cache hits, 70422 cache
size) in 0.28 seconds (maxdepth=8)

Move #8743, current score=235736
e634
9b22
7660
1111

Current scores: heur 234000, actual 239368
```

```
C:\Users\LENOVO\AppData\Local\Temp\HZ$D.368.318\HZ$D.368.319\2048-ai-master\20...

Move #8842, current score=239208
12e5
01ab
0043
0011

Current scores: heur 308000, actual 242884
Move 0: result 0.000000: eval'd 0 moves (0 cache hits, 0 cache size) in 0.00 seconds (maxdepth=0)
Move 1: result 344304.656250: eval'd 106252 moves (48443 cache hits, 25578 cache size) in 0.04 seconds (maxdepth=8)
Move 2: result 300641.500000: eval'd 86636 moves (32056 cache hits, 18935 cache size) in 0.03 seconds (maxdepth=8)
Move 3: result 314295.062500: eval'd 83556 moves (31848 cache hits, 18652 cache size) in 0.03 seconds (maxdepth=8)

Move #8843, current score=239208
10e5
00ab
0243
1111

Current scores: heur 276000, actual 242884
```

```
C:\Users\LENOVO\AppData\Local\Temp\HZ$D.368.318\HZ$D.368.319\2048-ai-master\20...

033b
1106

Current scores: heur 313000, actual 247296
Move 0: result 376421.000000: eval'd 1964756 moves (63258 cache hits, 55846 cache size) in 0.27 seconds (maxdepth=8)
Move 1: result 324696.125000: eval'd 1660672 moves (27015 cache hits, 33202 cache size) in 0.20 seconds (maxdepth=8)
Move 2: result 304382.406250: eval'd 200376 moves (23478 cache hits, 16759 cache size) in 0.03 seconds (maxdepth=8)
Move 3: result 381086.843750: eval'd 183460 moves (23946 cache hits, 16885 cache size) in 0.04 seconds (maxdepth=8)

Move #9112, current score=243576
04ae
0259
004b
0126

Current scores: heur 404000, actual 247336
Move 0: result 359135.687500: eval'd 479984 moves (43429 cache hits, 33193 cache size) in 0.07 seconds (maxdepth=8)
Move 1: result 348284.375000: eval'd 440780 moves (42480 cache hits, 32169 cache size) in 0.07 seconds (maxdepth=8)
```

```
C:\Users\LENOVO\AppData\Local\Temp\HZ$D.368.318\HZ$D.368.319\2048-ai-master\20...
4411
0110

Current scores: heur 284000, actual 268276
Move 0: result 352234.468750: eval'd 2032616 moves (49010 cache hits, 45336 cache size) in 0.31 seconds (maxdepth=8)
Move 1: result 296808.781250: eval'd 2080968 moves (25342 cache hits, 29675 cache size) in 0.30 seconds (maxdepth=8)
Move 2: result 363318.437500: eval'd 315960 moves (39215 cache hits, 28544 cache size) in 0.06 seconds (maxdepth=8)
Move 3: result 363793.562500: eval'd 244420 moves (37383 cache hits, 25118 cache size) in 0.05 seconds (maxdepth=8)

Move #9853, current score=264292
ea73
125c
0052
1002

Current scores: heur 324000, actual 268316
Move 0: result 360736.468750: eval'd 42024 moves (20550 cache hits, 10506 cache size) in 0.02 seconds (maxdepth=6)
Move 1: result 307411.312500: eval'd 42468 moves (21859 cache hits, 10617 cache size) in 0.02 seconds (maxdepth=6)
```

```
C:\Users\LENOVO\AppData\Local\Temp\HZ$D.368.318\HZ$D.368.319\2048-ai-master\20...
Move #10630, current score=280864
bc7e
0984
0031
0012

Current scores: heur 306000, actual 285252
Move 0: result 0.000000: eval'd 0 moves (0 cache hits, 0 cache size) in 0.00 seconds (maxdepth=0)
Move 1: result 299742.500000: eval'd 478056 moves (36344 cache hits, 28912 cache size) in 0.08 seconds (maxdepth=8)
Move 2: result 370426.375000: eval'd 429356 moves (40420 cache hits, 31965 cache size) in 0.07 seconds (maxdepth=8)
Move 3: result 0.000000: eval'd 0 moves (0 cache hits, 0 cache size) in 0.00 seconds (maxdepth=0)

Move #10631, current score=280864
bc7e
9840
3100
1201

Current scores: heur 347000, actual 285252
```

```
C:\Users\LENOVO\AppData\Local\Temp\HZ$D.368.318\HZ$D.368.319\2048-ai-master\20...
2121
2310
235e
d763

Current scores: heur 270000, actual 312576
Move 0: result 287378.812500: eval'd 1756620 moves (17863 cache hits, 24834 cache size) in 0.23 seconds (maxdepth=8)
Move 1: result 351676.625000: eval'd 2675376 moves (70103 cache hits, 67691 cache size) in 0.39 seconds (maxdepth=8)
Move 2: result 0.000000: eval'd 0 moves (0 cache hits, 0 cache size) in 0.00 seconds (maxdepth=0)
Move 3: result 299410.531250: eval'd 1324368 moves (18709 cache hits, 24497 cache size) in 0.18 seconds (maxdepth=8)

Move #11264, current score=307964
0021
2111
345e
d763

Current scores: heur 309000, actual 312600
Move 0: result 293095.968750: eval'd 2446436 moves (38141 cache hits, 45031 cache size) in 0.34 seconds (maxdepth=8)
```

```
C:\Users\LENOVO\AppData\Local\Temp\HZ$D.368.318\HZ$D.368.319\2048-ai-master\20...
0112
0124
034e
d862

Current scores: heur 346000, actual 313532
Move 0: result 292333.062500: eval'd 1152152 moves (32392 cache hits, 34964 cache size) in 0.19 seconds (maxdepth=8)
Move 1: result 354653.656250: eval'd 759292 moves (52622 cache hits, 42049 cache size) in 0.14 seconds (maxdepth=8)
Move 2: result 339752.968750: eval'd 1105684 moves (61251 cache hits, 58143 cache size) in 0.19 seconds (maxdepth=8)
Move 3: result 354197.125000: eval'd 699352 moves (66597 cache hits, 53944 cache size) in 0.13 seconds (maxdepth=8)

Move #11314, current score=308884
0012
1224
034e
d862

Current scores: heur 348000, actual 313536
Move 0: result 285991.218750: eval'd 1537880 moves (15674 cache hits, 18691 cache size) in 0.24 seconds (maxdepth=8)
```

```
C:\Users\LENOVO\AppData\Local\Temp\HZ$D.368.318\HZ$D.368.319\2048-ai-master\20...
Move #11699, current score=316688
0022
0003
1147
5dae

Current scores: heur 353000, actual 321480
Move 0: result 326240.593750: eval'd 258940 moves (47797 cache hits, 31650 cache
size) in 0.06 seconds (maxdepth=8)
Move 1: result 368607.562500: eval'd 234192 moves (43613 cache hits, 34007 cache
size) in 0.06 seconds (maxdepth=8)
Move 2: result 370125.343750: eval'd 39172 moves (19183 cache hits, 9793 cache s
ize) in 0.01 seconds (maxdepth=6)
Move 3: result 381494.562500: eval'd 40088 moves (19437 cache hits, 10022 cache
size) in 0.02 seconds (maxdepth=6)

Move #11700, current score=316700
0003
0003
1247
5dae

Current scores: heur 392000, actual 321492
```